

EXCERPT

Code sample: contractor payouts

An excerpt from our contractor module for Odoo 17: manifest, one model and one test. Full modules ship with docs and tests; all code is assigned to you or your client.

__manifest__.py

```
{
    "name": "Contractor Payouts",
    "version": "17.0.1.2.0",
    "summary": "Contractor rates, invoices from approved hours, payout runs",
    "author": "Evenmont",
    "license": "LGPL-3",
    "depends": ["hr_timesheet", "account", "project"],
    "data": [
        "security/ir.model.access.csv",
        "views/contractor_rate_views.xml",
        "views/payout_run_views.xml",
    ],
}
```

models/contractor_rate.py

```
from odoo import api, fields, models
from odoo.exceptions import ValidationError

class ContractorRate(models.Model):
    _name = "contractor.rate"
    _description = "Contractor hourly rate per project"
    _order = "date_from desc"

    partner_id = fields.Many2one("res.partner", required=True, index=True)
    project_id = fields.Many2one("project.project")
    rate = fields.Monetary(required=True)
    currency_id = fields.Many2one(
        "res.currency", default=lambda self: self.env.company.currency_id
    )
    date_from = fields.Date(required=True, default=fields.Date.context_today)

    @api.constrains("rate")
    def _check_rate(self):
        if any(r.rate <= 0 for r in self):
            raise ValidationError("A contractor rate must be positive.")

    def _rate_for(self, partner, project, day):
        """Most recent rate valid on the day: project-specific first, then default."""
        domain = [("partner_id", "=", partner.id), ("date_from", "<=", day)]
        for scope in ([("project_id", "=", project.id)], [("project_id", "=", False)]):
            rate = self.search(domain + scope, limit=1)
            if rate:
                return rate.rate
        return 0.0
```

tests/test_contractor_rate.py

```
from odoo.tests import TransactionCase, tagged

@tagged("post_install", "-at_install")
class TestContractorRate(TransactionCase):
    @classmethod
    def setUpClass(cls):
        super().setUpClass()
        cls.contractor = cls.env["res.partner"].create({"name": "Field Contractor"})
        cls.project = cls.env["project.project"].create({"name": "Riverside survey"})
        Rate = cls.env["contractor.rate"]
        Rate.create({"partner_id": cls.contractor.id, "rate": 60, "date_from": "2026-01-01"})
        Rate.create({"partner_id": cls.contractor.id, "project_id": cls.project.id,
                     "rate": 75, "date_from": "2026-03-01"})

    def test_project_rate_wins(self):
        rate = self.env["contractor.rate"]._rate_for(self.contractor, self.project, "2026-03-10")
        self.assertEqual(rate, 75)

    def test_default_rate_before_project_rate(self):
        rate = self.env["contractor.rate"]._rate_for(self.contractor, self.project, "2026-02-10")
        self.assertEqual(rate, 60)
```

How we work on code

- ✓ Your Git repository, your review rules, pull requests for every change.
- ✓ Tests for business logic; linting with the OCA guidelines.
- ✓ Technical documentation for every module, handed over with the code.
- ✓ No lock-in: the code is assigned to you or your client.